

# FULL STACK DEVELOPMENT

## Unit 1: Introduction to Web

---

### 1. Introduction to Web

---

#### 1.1 What is a Website?

A **website** is a collection of web pages that are stored on the internet and can be accessed using a web browser such as Google Chrome, Firefox, or Edge. Think of it like a digital book — just as a book has many pages, a website has many web pages linked together. Each web page can have text, images, videos, links, and other content. You visit websites every day — when you open Google to search for something or visit Amazon to buy a product, you are visiting a website.

Every website has a unique address called a **URL (Uniform Resource Locator)**. For example, <https://www.google.com> is the URL of Google's website. When you type this address in your browser, your browser finds and opens that website for you.

#### 1.2 What is Web Development?

**Web Development** is the process of building and creating websites and web applications. Just like a civil engineer builds houses and buildings, a web developer builds websites. Web development involves writing code using special programming languages that tell the browser how to display content and how the website should work.

Web development is divided into two main parts:

- **Frontend Development:** This is the part of the website that you can see and interact with — the design, buttons, colors, images, and text. Frontend developers use HTML, CSS, and JavaScript.
- **Backend Development:** This is the part that works behind the scenes — the server, database, and application logic. Backend developers use languages like Node.js, Python, or Java.
- **Full Stack Development:** A full stack developer can work on both frontend and backend. This course teaches you to become a full stack developer.

## 1.3 Types of Websites

There are two main types of websites you should know about:

| Type            | Description & Example                                                                                                                              |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Static Website  | Every user sees the same content. The pages do not change. Example: A simple company brochure website or a personal portfolio.                     |
| Dynamic Website | Content changes based on the user or the data. Example: Amazon shows different products to different users. Facebook shows your personal newsfeed. |

Most modern websites like Google, Amazon, and Instagram are **dynamic websites** because they change content based on user input and data stored in databases.

## 1.4 Importance of Web Development in Real Life

Web development has become one of the most important skills in today's world. Almost every business needs a website to reach its customers. Here are some real-life examples that show why web development matters:

- Amazon uses web development to sell millions of products online through its e-commerce platform.
- Google uses web development to provide its powerful search engine to billions of users.
- YouTube uses web development to allow users to upload and watch videos from anywhere in the world.
- Online banking apps like SBI or HDFC use web development to let customers check balance and transfer money without visiting a bank.

Learning web development opens the door to high-paying career opportunities and allows you to create your own applications and products.

## 2. Client–Server Architecture

### 2.1 What is Client–Server Architecture?

**Client–Server Architecture** is a model of communication where two computers work together — one called the **Client** and the other called the **Server**. The client sends a request for information, and the server receives that request, processes it, and sends back a response. This is how almost all websites and web applications work.

### 2.2 What is a Client?

A **client** is any device or application that requests services or resources from a server. In web development, the client is usually a **web browser** (like Chrome or Firefox) running on your computer, tablet, or smartphone. The client is also called the **"frontend"** — it is what the user sees and interacts with.

For example, when you open your browser and visit [www.google.com](http://www.google.com), your browser is acting as the client. It sends a request to Google's server saying, "Please give me the Google homepage."

### 2.3 What is a Server?

A **server** is a powerful computer or program that stores data, files, and applications and makes them available to clients over the internet. The server is also called the **"backend"**. It runs all the time, waiting for requests from clients. When it receives a request, it processes it and sends back the answer (called a **response**).

For example, Google's servers store the entire search engine. When you search for something, their server processes your search and sends back the list of results.

### 2.4 Real-Life Analogy: Customer and Restaurant

#### Analogy: Restaurant

Think of a restaurant as a Client-Server system:

- You (the CUSTOMER) = Client — you place an order
- The WAITER = HTTP Protocol — carries messages between you and kitchen
- The KITCHEN = Server — receives the order, prepares the food, sends it back
- The FOOD = Response — what the server returns to you

Just like in a restaurant, the client (you) never goes into the kitchen. You only communicate through the waiter (HTTP). The kitchen (server) processes your order and delivers the result.

## 2.5 Diagram Explanation: Client–Server Communication

### Client-Server Architecture

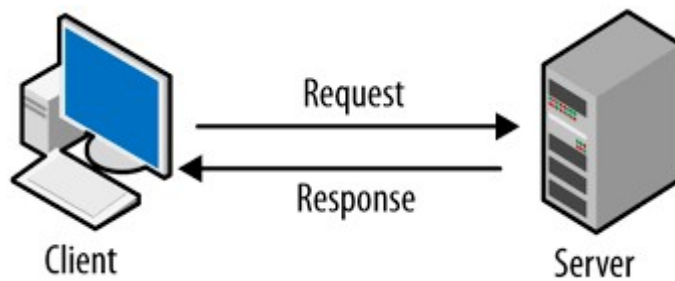
Step 1: User types a URL in the browser (Client)

Step 2: Browser sends an HTTP Request to the Server

Step 3: Server receives the request and processes it

Step 4: Server sends back an HTTP Response (HTML page)

Step 5: Browser displays the webpage to the user



## 3. Communication Protocol (HTTP)

### 3.1 What is HTTP?

**HTTP** stands for **HyperText Transfer Protocol**. It is the set of rules (protocol) that defines how messages are sent and received between a client (browser) and a server over the internet. Think of HTTP as a **common language** that both the client and server understand and use to communicate with each other.

When you see a URL starting with *http://* or *https://*, the **"S" in HTTPS stands for Secure** — it means the communication is encrypted and protected. Modern websites use HTTPS to keep your data safe.

### 3.2 Request and Response Concept

Every interaction on the web follows a simple Request-Response cycle. Understanding this concept is very important for web development.

| HTTP Request                              | HTTP Response                                  |
|-------------------------------------------|------------------------------------------------|
| Sent BY the Client (Browser)              | Sent BY the Server                             |
| Contains: Method, URL, Headers, Body      | Contains: Status Code, Headers, Body           |
| Example: "Give me the homepage of Amazon" | Example: "Here is the HTML of Amazon homepage" |

### 3.3 Structure of an HTTP Request

An HTTP Request has the following important parts:

- **Method:** Tells the server what type of action to perform (GET, POST, etc.)
- **URL:** The address of the resource being requested (e.g., /home, /products)
- **Headers:** Extra information sent with the request (e.g., browser type, language)
- **Body (optional):** Data sent with the request (used in POST requests, like form data)

### 3.4 HTTP Methods: GET and POST

The two most commonly used HTTP methods that you need to know are GET and POST:

| GET Method                             | POST Method                                 |
|----------------------------------------|---------------------------------------------|
| Used to FETCH or READ data from server | Used to SEND or SUBMIT data to server       |
| Data is visible in the URL             | Data is hidden (sent in request body)       |
| Example: Searching on Google           | Example: Submitting a login form            |
| Safe to use for reading data           | Used when data should not be visible in URL |
| URL: /search?query=html                | URL: /login (data in body)                  |

## 3.5 HTTP Status Codes

After the server processes a request, it sends back a response with a status code that tells the client what happened. Here are the most important ones:

| Status Code               | Meaning                                                          |
|---------------------------|------------------------------------------------------------------|
| 200 OK                    | Request was successful! The server found what you asked for.     |
| 301 Moved Permanently     | The page has moved to a new URL permanently.                     |
| 404 Not Found             | The page or resource you requested does not exist on the server. |
| 500 Internal Server Error | Something went wrong on the server side.                         |

## 3.6 Real-Life Example: Ordering Food Online

|                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------|
|  <b>Real-Life Example: Online Food Order</b> |
| Imagine ordering food on Zomato or Swiggy:                                                                                    |
| 1. You open Zomato on your phone (Client opens browser)                                                                       |
| 2. You search for 'Pizza' (Client sends GET request: /search?food=pizza)                                                      |
| 3. Zomato server receives your request and searches its database                                                              |
| 4. Server sends back a list of pizza restaurants (Response with Status 200 OK)                                                |
| 5. You place an order by filling in your name and address (POST request with form data)                                       |
| 6. Server receives your order details and confirms it (Response: Order Confirmed!)                                            |
|                                                                                                                               |
| This shows how HTTP GET is used to fetch data and HTTP POST is used to send data.                                             |

## 4. Structure of HTML Documents

### 4.1 What is HTML?

**HTML** stands for **HyperText Markup Language**. It is the basic building block of every webpage. HTML is not a programming language — it is a **markup language**, which means it uses special tags to define the structure and content of a webpage. Think of HTML as the **skeleton or frame of a building** — it defines what goes where on the page (headings, paragraphs, images, links, etc.).

Just like how you use headings and paragraphs when writing a document in Word, HTML uses tags to organize content on a webpage. Every webpage you visit is made of HTML.

### 4.2 Basic Structure of an HTML Document

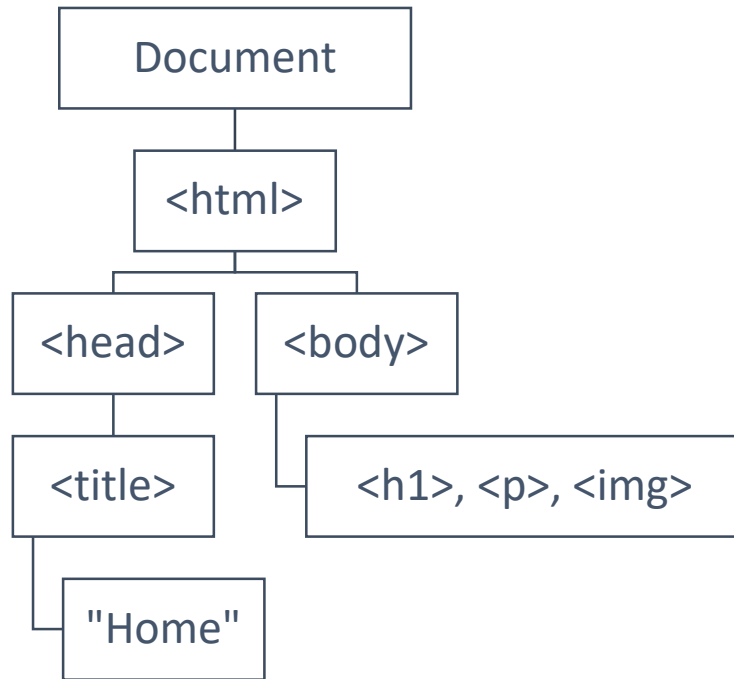
Every HTML document follows a specific structure with required tags. Let us look at each part:

```
<!DOCTYPE html>
<html>
  <head>
    <title>My First Webpage</title>
  </head>
  <body>
    <h1>Welcome to Web Development!</h1>
    <p>This is my first HTML page.</p>
  </body>
</html>
```

Let us understand what each part means:

- **<!DOCTYPE html>:** Tells the browser that this is an HTML5 document. Always write this at the very top.
- **<html>:** This is the root tag — it wraps all the content of the entire HTML page.
- **<head>:** Contains information about the page that is NOT displayed on screen, like the page title and linked CSS files.
- **<title>:** Sets the title of the webpage, which appears on the browser tab at the top.
- **<body>:** Contains all the content that is actually **VISIBLE** on the webpage — text, images, links, etc.

## 4.3 Diagram Explanation: Tree Structure of HTML



### HTML Document Tree Structure

- The Document is at the top (root)
- `<html>` is the parent of everything
- `<head>` and `<body>` are children of `<html>`
- `<title>` is inside `<head>`
- All visible content (`h1`, `p`, `img`) is inside `<body>`



## 5. Basic HTML Markup Tags

### 5.1 Heading Tags (h1 to h6)

HTML provides **6 levels of headings** from `<h1>` to `<h6>`. `<h1>` is the largest and most important heading (like a chapter title), and `<h6>` is the smallest. You should use headings to organize your content, just like how a book has main headings and sub-headings.

```
<h1>This is Heading 1 - Largest</h1>
<h2>This is Heading 2</h2>
<h3>This is Heading 3</h3>
<h4>This is Heading 4</h4>
<h5>This is Heading 5</h5>
<h6>This is Heading 6 - Smallest</h6>
```

### 5.2 Paragraph Tag (p)

The `<p>` tag is used to write a block of text (paragraph) on a webpage. The browser automatically adds space before and after a paragraph, making it easy to read. Every time you want to write a new paragraph, use a new `<p>` tag.

```
<p>This is my first paragraph. HTML is easy to learn!</p>
<p>This is my second paragraph. Web development is fun.</p>
```

### 5.3 Image Tag (img)

The `<img>` tag is used to display an image on a webpage. It is a **self-closing tag** — it does not have a closing tag. It requires two important attributes: **src** (the source path of the image) and **alt** (alternative text shown if the image cannot load, also important for accessibility).

```
<!-- Display an image from a file -->


<!-- Display an image from the internet -->

```

## 5.4 Link Tag (a)

The `<a>` tag (anchor tag) is used to create clickable links that take you to another page or website. The most important attribute is **href** (Hypertext Reference), which contains the URL where the link should go. By default, links open in the same tab, but you can use **target="\_blank"** to open in a new tab.

```
<!-- Link to an external website -->
<a href="https://www.google.com">Visit Google</a>

<!-- Link that opens in a new tab -->
<a href="https://www.amazon.in" target="_blank">Open Amazon</a>

<!-- Link to another page in the same website -->
<a href="contact.html">Contact Us</a>
```

## 5.5 List Tags (ul, ol, li)

HTML provides two types of lists:

- **Unordered List (<ul>):** Shows items with bullet points (dots). Used when order does not matter.
- **Ordered List (<ol>):** Shows items with numbers (1, 2, 3...). Used when order matters.
- **List Item (<li>):** Each item in both types of lists is wrapped in an `<li>` tag.

```
<!-- Unordered List (bullet points) -->
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>

<!-- Ordered List (numbered) -->
<ol>
  <li>First learn HTML</li>
  <li>Then learn CSS</li>
  <li>Then learn JavaScript</li>
</ol>
```

## 6. Introduction to CSS

---

### 6.1 What is CSS?

**CSS** stands for **Cascading Style Sheets**. CSS is used to control the visual appearance (design) of HTML elements — the colors, fonts, sizes, spacing, and layout of a webpage. While HTML creates the structure, CSS makes it beautiful.

#### Analogy: HTML vs CSS

Think of building a house:

- HTML = The structure (walls, doors, windows, roof)
- CSS = The interior design (paint color, furniture, flooring, decorations)

HTML alone creates a plain, unstyled page. CSS transforms it into a beautiful, professional website.

### 6.2 Three Ways to Apply CSS

There are three methods to add CSS to an HTML page:

#### Method 1: Inline CSS

Inline CSS is written directly inside an HTML tag using the style attribute. It only affects that single element.

```
<p style="color: red; font-size: 18px;">This text is red!</p>
```

#### Method 2: Internal CSS

Internal CSS is written inside a <style> tag within the <head> section. It applies to the entire page.

```
<head>
  <style>
    p {
      color: blue;
      font-size: 16px;
    }
  </style>
</head>
```

### Method 3: External CSS

External CSS is written in a separate file (style.css) and linked to the HTML file using a <link> tag. This is the best practice for real projects because you can reuse the same CSS for multiple pages.

```
<!-- In the HTML file (index.html) -->
<head>
  <link rel="stylesheet" href="style.css">
</head>

/* In the CSS file (style.css) */
p {
  color: green;
  font-size: 16px;
}
```

| Method       | Best Use                                                                        |
|--------------|---------------------------------------------------------------------------------|
| Inline CSS   | Quick fixes; single element only. Not recommended for large projects.           |
| Internal CSS | Single page styling. Useful for small, one-page websites.                       |
| External CSS | Best practice. Use this for all real projects. One CSS file for multiple pages. |

## 7. Working with Text and Images using CSS

### 7.1 Styling Text with CSS

CSS gives you full control over how text looks on your webpage. Here are the most important text properties:

```
p {  
  color: navy;           /* text color */  
  font-size: 18px;       /* text size */  
  font-family: Arial;    /* font type */  
  font-weight: bold;     /* makes text bold */  
  text-align: center;    /* center the text */  
  text-decoration: underline; /* underlines text */  
  line-height: 1.6;      /* space between lines */  
}
```

| CSS Property | What It Does                                                     |
|--------------|------------------------------------------------------------------|
| color        | Changes the text color. Example: color: red;                     |
| font-size    | Sets the size of the text. Example: font-size: 20px;             |
| font-family  | Changes the font style. Example: font-family: Arial;             |
| font-weight  | Makes text bold or normal. Example: font-weight: bold;           |
| text-align   | Aligns text left, right, or center. Example: text-align: center; |

## 7.2 Styling Images with CSS

You can also use CSS to control the size, shape, and style of images on your webpage:

```
img {  
  width: 300px;           /* set image width */  
  height: 200px;          /* set image height */  
  border: 2px solid black; /* add border around image */  
  border-radius: 10px;     /* rounds the corners */  
  opacity: 0.8;           /* makes image slightly transparent */  
}
```

An important tip: Using **width: 100%** makes an image responsive — it stretches or shrinks to fit the screen size. This is very useful for mobile-friendly websites.

## 8. CSS Selectors

---

### 8.1 What is a Selector?

A **CSS Selector** is a pattern used to select or target specific HTML elements so that CSS styles can be applied to them. Think of a selector as a **"pointing finger"** — it tells CSS which element you want to style. Without selectors, CSS would not know which element to apply the style to.

### 8.2 Types of CSS Selectors

#### Type 1: Element Selector

The **Element Selector** selects all HTML elements of a specific type (tag name). For example, if you write `p { color: red; }`, ALL paragraphs on the page become red.

```
/* All <p> elements will have blue text */
p {
  color: blue;
}

/* All <h1> elements will have a font size of 36px */
h1 {
  font-size: 36px;
}
```

#### Type 2: Class Selector

The **Class Selector** uses a **dot (.)** followed by the class name. A class can be applied to multiple elements, and one element can have multiple classes. This is the most commonly used selector in real projects.

```
/* CSS: Select elements with class 'highlight' */
.highlight {
  background-color: yellow;
  font-weight: bold;
}

<!-- HTML: Apply class to elements -->
<p class="highlight">This paragraph is highlighted!</p>
<h2 class="highlight">This heading is also highlighted!</h2>
```

### Type 3: ID Selector

The **ID Selector** uses a **hash symbol (#)** followed by the ID name. An ID must be **unique** — only ONE element on a page should have a particular ID. Use IDs for styling elements that appear only once on a page.

```
/* CSS: Select element with id 'main-title' */
#main-title {
  color: darkblue;
  font-size: 40px;
  text-align: center;
}
```

```
<!-- HTML: Only one element should have this ID -->
<h1 id="main-title">Welcome to My Website</h1>
```

| Selector | Symbol  | Usage                             | Example                      |
|----------|---------|-----------------------------------|------------------------------|
| Element  | None    | Targets all elements of that type | p { color: red; }            |
| Class    | .(dot)  | Targets elements with that class  | .box { width: 100px; }       |
| ID       | #(hash) | Targets one unique element        | #header { font-size: 30px; } |



## 9. CSS Flexbox

---

### 9.1 What is Flexbox?

**CSS Flexbox** (Flexible Box Layout) is a powerful layout system in CSS that makes it **easy to arrange elements in a row or column** and control how they are positioned and spaced. Before Flexbox, creating layouts in CSS was very difficult. Flexbox simplifies this greatly.

Think of Flexbox like a tray of items at a cafeteria — you can arrange the trays in a row, decide whether to put them close together or spread them out, and align them to the top, middle, or bottom of the shelf.

### 9.2 Basic Flexbox Properties

#### display: flex

To use Flexbox, you must first set **display: flex** on the **container** (parent element). This turns the container into a flex container, and all its direct children become **flex items** that follow flex rules.

```
.container {  
  display: flex;    /* Activates Flexbox */  
}
```

#### justify-content

**justify-content** controls how flex items are arranged **horizontally (along the main axis)**. Common values are:

- **flex-start**: Items align to the left (default)
- **flex-end**: Items align to the right
- **center**: Items are centered horizontally
- **space-between**: Items spread out with space between them, no space at edges
- **space-around**: Items spread out with equal space around each

#### align-items

**align-items** controls how flex items are arranged **vertically (along the cross axis)**. Common values: **flex-start**, **flex-end**, **center**.

## 9.3 Simple Layout Example

```
/* CSS */
.container {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background-color: lightblue;
  padding: 10px;
}
.box {
  width: 100px;
  height: 100px;
  background-color: navy;
  color: white;
  text-align: center;
}

<!-- HTML -->
<div class="container">
  <div class="box">Box 1</div>
  <div class="box">Box 2</div>
  <div class="box">Box 3</div>
</div>
```

# 10. JavaScript Basics

## 10.1 What is JavaScript?

**JavaScript (JS)** is a **programming language** used to make web pages interactive and dynamic. While HTML defines the structure and CSS defines the design, JavaScript adds **behavior and interactivity** to the webpage. For example, when you click a button and something happens (a menu opens, a form is validated, a popup appears), that is JavaScript doing the work.

| Technology | Role                     | Example                                    |
|------------|--------------------------|--------------------------------------------|
| HTML       | Structure / Skeleton     | Defines headings, paragraphs, buttons      |
| CSS        | Design / Appearance      | Colors, fonts, layout, animations          |
| JavaScript | Behavior / Interactivity | Button clicks, form validation, AJAX calls |

## 10.2 Data Types in JavaScript

Every value in JavaScript has a data type. The three basic data types you need to know are:

- **Number:** Represents numeric values. Example: 10, 3.14, -5
- **String:** Represents text. Always written inside quotes. Example: "Hello", 'Web Dev'
- **Boolean:** Represents true or false values. Used in conditions. Example: true, false

## 10.3 Variables in JavaScript

A **variable** is a container used to store data values. In JavaScript, you can declare variables using three keywords: **var**, **let**, or **const**

```
var name = "Raj";           // Old way, avoid using this
let age = 21;                // Modern way, value can change
const PI = 3.14159;          // Value CANNOT be changed (constant)

console.log(name);           // Output: Raj
console.log(age);            // Output: 21
```

| Keyword | Can Value Change? | Best Use                                     |
|---------|-------------------|----------------------------------------------|
| var     | Yes               | Old code. Avoid using in new projects.       |
| let     | Yes               | Use when the variable value needs to change. |
| const   | No                | Use when the value should remain fixed.      |

## 10.4 Functions in JavaScript

A **function** is a block of code that performs a specific task. You define the function once and can **call (execute)** it as many times as needed. Functions help avoid code repetition. Think of a function like a **recipe** — you write the recipe once and can make the dish many times.

```
// Define a function
function greet(studentName) {
  console.log("Hello, " + studentName + "!");
}

// Call the function
greet("Anita");      // Output: Hello, Anita!
greet("Ravi");       // Output: Hello, Ravi!
```

# 11. JavaScript Events

## 11.1 What is an Event?

An **event** is an action that happens in the browser — like a user clicking a button, typing in an input box, hovering over an element, or submitting a form. JavaScript can **detect and respond to these events**, which is what makes web pages interactive.

Think of events like a doorbell — when someone presses the bell (event), you hear it and open the door (event handler). In JavaScript, you write functions (event handlers) that run when a specific event occurs.

## 11.2 Common Events

| Event     | When It Occurs                   | HTML Attribute |
|-----------|----------------------------------|----------------|
| click     | When user clicks on an element   | onclick        |
| change    | When value of input changes      | onchange       |
| mouseover | When mouse hovers over element   | onmouseover    |
| keypress  | When user presses a keyboard key | onkeypress     |
| submit    | When a form is submitted         | onsubmit       |

## 11.3 Button Click Example

```
<!DOCTYPE html>
<html>
<body>
  <button onclick="showMessage()">Click Me!</button>
  <p id="message"></p>

  <script>
    function showMessage() {
      document.getElementById("message").innerHTML
        = "Hello! You clicked the button!";
    }
  </script>

</body>
</html>
```

## 12. Introduction to AJAX

### 12.1 What is AJAX?

**AJAX** stands for **Asynchronous JavaScript and XML**. AJAX is a technique in web development that allows a web page to **communicate with a server and update content without reloading the entire page**. This makes web applications faster and more user-friendly.

The word 'Asynchronous' means the browser does not have to wait and stop everything while waiting for a server response. It can continue showing the page to the user while the data is being fetched in the background.

### 12.2 The Problem AJAX Solves

Before AJAX, every time you needed new data from the server, the entire page had to reload. Imagine:

- You are on Facebook and you like a post. Without AJAX, the ENTIRE page would have to refresh!
- On Gmail, every time you receive a new email, the page would reload completely.
- On a shopping site, every time you add to cart, the page reloads from scratch.

AJAX solved this problem by allowing web pages to fetch only the specific data they need and update only that part of the page.

### 12.3 Normal Request vs AJAX Request

| Normal HTTP Request                    | AJAX Request                               |
|----------------------------------------|--------------------------------------------|
| Entire page reloads                    | Only specific part of page updates         |
| User sees a blank screen during load   | User experience is smooth, no interruption |
| Slow and less user-friendly            | Fast and interactive                       |
| Example: Old websites from early 2000s | Example: Facebook, Gmail, Google Maps      |

### 12.4 Real-Life AJAX Example

|                                                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------|
|  <b>AJAX in Google Search (Autocomplete)</b> |
| When you type in Google's search box:                                                                                           |
| 1. You type the letter 'P'                                                                                                      |
| 2. JavaScript sends an AJAX request to Google's server: 'Give me suggestions for P'                                             |
| 3. Google's server responds with suggestions: Pizza, Python, Paris...                                                           |
| 4. JavaScript updates ONLY the dropdown list — the rest of the page stays the same                                              |
| 5. No page reload happens!                                                                                                      |
| This is AJAX in action — fetching data from a server in the background and updating part of the page.                           |

## 13. AJAX GET and POST

### 13.1 AJAX GET Request

An **AJAX GET request** is used to **fetch or retrieve data** from a server without reloading the page. In modern JavaScript, we use the **fetch()** function to make AJAX requests. The **fetch()** function sends a request to a URL and returns a Promise — an object that represents the future result.

```
// AJAX GET request using fetch()
fetch('https://api.example.com/students')
  .then(function(response) {
    return response.json(); // Convert to JSON format
  })
  .then(function(data) {
    console.log(data); // Use the received data
    document.getElementById('output').innerHTML
      = JSON.stringify(data);
  })
  .catch(function(error) {
    console.log('Error:', error); // Handle errors
  });
```

### 13.2 AJAX POST Request

An **AJAX POST request** is used to **send data** to the server. For example, submitting a registration form, posting a comment, or placing an order. With AJAX POST, the data is sent without the page reloading.

```
// AJAX POST request using fetch()
fetch('https://api.example.com/login', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    username: 'raj123',
    password: 'secure@pass'
  }) })
  .then(function(response) { return response.json(); })
  .then(function(data) { console.log('Login result:', data); })
  .catch(function(error) { console.log('Error:', error); });
```

| AJAX GET                       | AJAX POST                             |
|--------------------------------|---------------------------------------|
| Fetches/reads data from server | Sends/submits data to server          |
| No data sent in request body   | Data sent in request body (JSON/form) |
| Example: Load user profile     | Example: Submit login form            |
| Server does not change data    | Server stores or processes data       |



## 14. Integration Example: HTML + CSS + JavaScript

### 14.1 How All Three Work Together

In a real web project, **HTML**, **CSS**, and **JavaScript** always work together as a team. HTML creates the structure, CSS designs the look, and JavaScript adds interactive behavior. Let us look at a complete example that uses all three to create a simple interactive page.

```
<!DOCTYPE html>
<html>
<head>
  <title>My Web Page</title>
  <style>
    /* CSS: Styling the page */
    body { font-family: Arial; background-color: #f0f4f8; }
    h1 { color: #1A3A5C; text-align: center; }
    .container {
      display: flex;
      justify-content: center;
      align-items: center;
      padding: 20px;
    }
    #greetBox {
      background-color: white;
      border: 2px solid #2E75B6;
      padding: 20px;
      border-radius: 10px;
      text-align: center;
    }
    button {
      background-color: #2E75B6;
      color: white;
      border: none;
      padding: 10px 20px;
      border-radius: 5px;
      cursor: pointer;
    }
    #result { color: green; font-size: 20px; margin-top: 10px; }
```

```
</style>
</head>
<body>

  <!-- HTML: Structure -->
  <h1>Full Stack Web Page Demo</h1>
  <div class="container">
    <div id="greetBox">
      <input type="text" id="nameInput" placeholder="Enter your
name">
      <br><br>
      <button onclick="greetUser()">Say Hello!</button>
      <p id="result"></p>
    </div>
  </div>

  <script>
    // JavaScript: Behavior
    function greetUser() {
      var name = document.getElementById('nameInput').value;
      if (name !== '') {
        document.getElementById('result').innerHTML
          = 'Hello, ' + name + '! Welcome to Web Dev!';
      } else {
        document.getElementById('result').innerHTML
          = 'Please enter your name first!';
      }
    }
  </script>

</body>
</html>
```